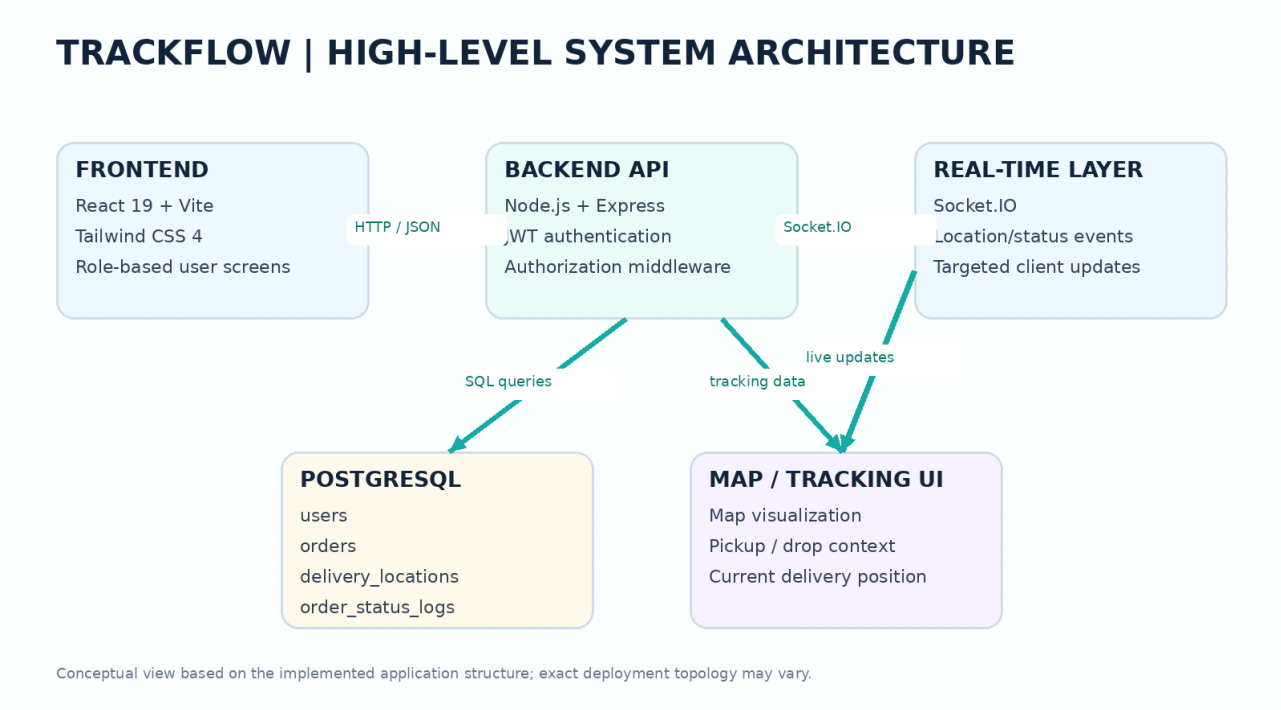
FULL STACK WEB DEVELOPMENT

PROJECT REPORT

TRACKFLOW

Real-Time Delivery Tracking and Order Management System

A full-stack web application for managing delivery orders, coordinating delivery agents, and presenting live delivery progress to customers.



PROJECT DETAIL	VALUE
Frontend	React 19, Vite, Tailwind CSS 4
Backend	Node.js, Express.js, Socket.IO
Database	PostgreSQL
Repository	https://github.com/itsurvey6-lab/TrackFlow-RealTime-Tracking-Fullstack-project

Submitted By: ABDUL FAYAS VK

Submitted To: Unified Mentor

Document Information

This report describes the TrackFlow application based on the project scope and implementation details available for this report. Personal, institutional, and supervisor details are intentionally left blank for completion by the author.

Item	Details
Project title	TrackFlow – Real-Time Delivery Tracking and Order Management System
Document type	Detailed Software Project Report
Version	1.0
Application status	Core application workflows, real-time tracking, and map display verified locally by the project author
Deployment	Cloud deployment is outside this report's current scope and is planned as a later activity
Source repository	https://github.com/itsurvey6-lab/TrackFlow-RealTime-Tracking-Fullstack-project

Table of Contents

- 1. Executive Summary
- 2. Introduction
- 3. Problem Statement
- 4. Objectives and Scope
- 5. Technology Stack
- 6. System Architecture
- 7. Database Design
- 8. User Roles and Modules
- 9. Order Lifecycle and Business Workflow
- 10. Real-Time Tracking with Socket.IO
- 11. Map and Location Visualization
- 12. Backend API and Application Structure
- 13. Security and Access Control
- 14. Testing and Verification
- 15. Challenges and Learning Outcomes
- 16. Limitations and Future Enhancements
- 17. Conclusion
- 18. References

1. Executive Summary

TrackFlow is a full-stack delivery tracking and order management application designed to coordinate customers, administrators, and delivery agents. It brings order creation, agent assignment, delivery status management, location updates, and customer-facing tracking into one web application.

The frontend is built with React and Vite. The backend uses Node.js and Express.js, with PostgreSQL providing persistent relational storage. Socket.IO supports real-time communication for delivery progress and location updates. The application includes role-aware screens for administrators, customers, and delivery agents, together with order management, customer management, notifications, reports, and a tracking interface.

The core workflows, real-time tracking, and map display have been completed and verified in the local development environment by the project author. Public cloud deployment is treated as a subsequent release activity and is not represented as completed in this report.

2. Introduction

Delivery operations involve several participants and a sequence of events. Customers need to know whether an order has been accepted, assigned, collected, and delivered. Administrators need a clear view of orders and delivery resources. Delivery agents need access to assigned work and a way to communicate progress and location.

TrackFlow addresses these needs through a centralized web application. A customer can create and follow an order; an administrator can manage orders and assign delivery agents; and an agent can progress an assigned order through the delivery lifecycle.

3. Problem Statement

When order handling, agent coordination, and customer updates are performed through disconnected tools or manual communication, information can become delayed or inconsistent. Customers may not know the current status of an order, administrators may have limited visibility into active deliveries, and delivery progress may not be recorded in a consistent history.

The project aims to provide a single application in which order records, delivery assignments, status history, and location information are connected and made available to the appropriate user roles.

4. Objectives and Scope

4.1 Objectives

- Provide a web interface for customers to create and monitor delivery orders.
- Provide administrative functions for viewing and managing orders and assigning delivery agents.
- Provide delivery agents with access to assigned deliveries and status progression.
- Store operational records in PostgreSQL.
- Use real-time communication to distribute relevant delivery status and location updates.
- Display delivery location information in the tracking interface.
- Apply authentication and role-based authorization.

4.2 Project Scope

The implemented scope includes customer, administrator, and delivery-agent workflows; order lifecycle management; location storage and display; real-time updates; notification functionality; and supporting management/reporting screens. The report focuses on the application as developed and verified locally. Cloud hosting and production operations are not claimed as completed deliverables.

5. Technology Stack

Layer	Technology	Purpose
Frontend	React 19	Component-based user interface
Frontend tooling	Vite	Development server and build tooling
Styling	Tailwind CSS 4 and CSS	Responsive layout and styling
Routing	React Router	Client-side page navigation
Backend	Node.js and Express.js	Server runtime, API routes, and middleware
Real-time	Socket.IO	Bidirectional event communication
Database	PostgreSQL	Relational data storage
Database access	pg	Backend database connectivity
Authentication	JWT and bcrypt	Token authentication and password hashing
Development	npm	Dependency and script management

6. System Architecture

TrackFlow follows a client-server architecture. The React frontend communicates with the Express backend over HTTP for normal application operations. Socket.IO provides a real-time communication channel for updates that should be reflected without requiring a page refresh. The backend reads and writes application data in PostgreSQL.

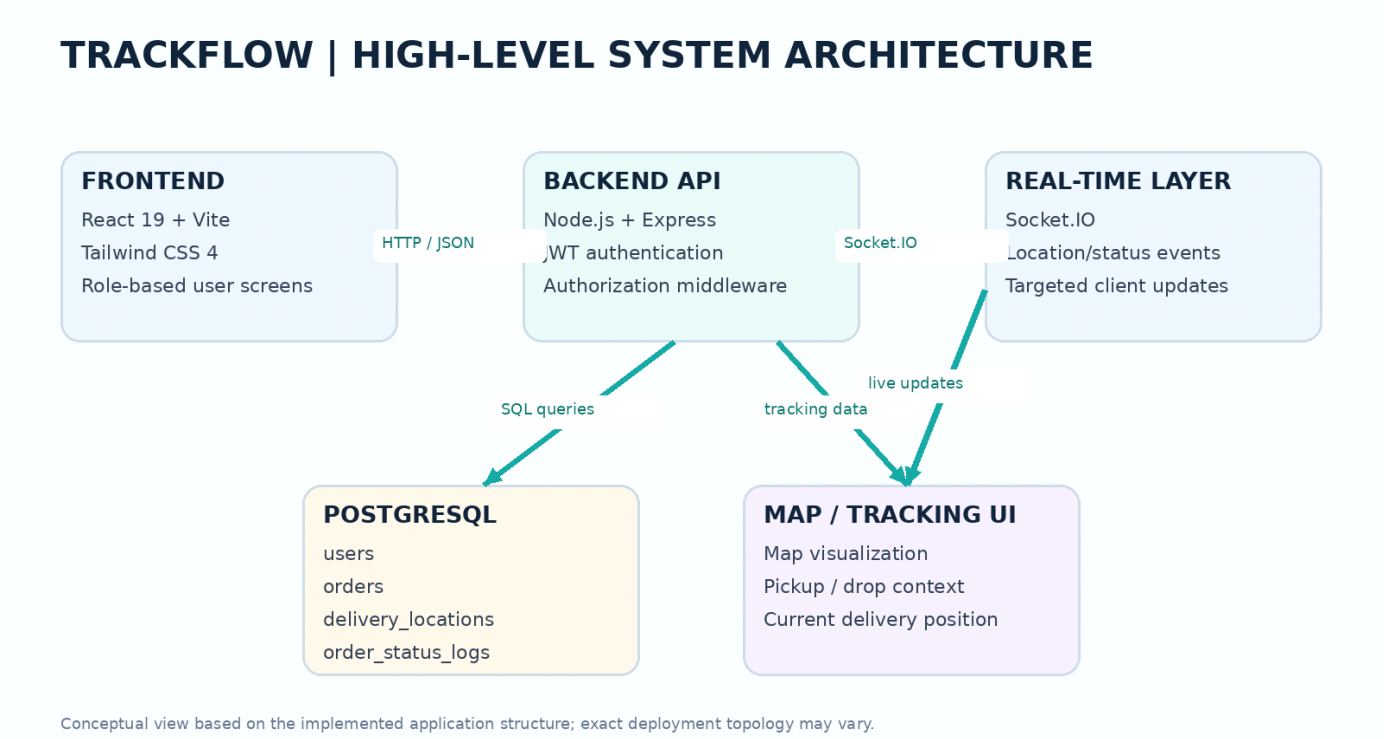


Figure 1. Conceptual high-level architecture of TrackFlow.

6.1 Main Components

- Frontend: role-specific pages, forms, order lists, dashboards, and tracking interface.
- Backend API: request validation, business rules, authentication, authorization, and data operations.
- Real-time layer: sends relevant delivery events to connected clients using Socket.IO.
- PostgreSQL: stores users, orders, delivery locations, and order status history.
- Tracking interface: presents order progress and location information.

6.2 Typical Request Flow

A user action begins in the frontend and is sent to the backend. The backend validates the request and checks identity and role. If valid, relevant database records are read or updated. For real-time changes, the backend emits an event to the relevant connected clients.

7. Database Design

PostgreSQL is the primary relational database. Four core tables identified during development are users, orders, delivery_locations, and order_status_logs. These separate account identity, order details, location observations, and the history of order-state changes.

Table	Responsibility	Typical relationship
users	Account identity, login information, role, metadata.	A user may be customer, administrator, or delivery agent.
orders	Delivery order information and assignment/status details.	References customer and assigned delivery agent.
delivery_locations	Location observations associated with a delivery/order.	Related to the order and/or delivery agent.
order_status_logs	History of order status changes and timestamps.	Multiple log entries can belong to one order.

7.1 User Roles

Delivery agents are represented as users with the delivery_agent role rather than requiring a separate delivery-agent account table. This centralizes authentication and account management.

7.2 Order Status History

The status log provides an auditable sequence of lifecycle changes. The workflow exercised during development included placed, assigned, picked_up, in_transit, and delivered states.

7.3 Data Integrity

- Use primary keys to identify records uniquely.
- Use foreign keys to connect orders with customers and assigned agents.
- Store timestamps for order creation and status/location events.
- Validate status transitions in backend logic.
- Restrict data access according to role and ownership.

8. User Roles and Application Modules

Role	Responsibilities	Capabilities
Customer	Create and follow delivery orders.	Register/login, create orders, view own orders, follow status and location.
Administrator	Coordinate delivery operations.	View/manage orders, assign agents, manage customer information, view reports.
Delivery Agent	Complete assigned deliveries.	View assigned orders, update delivery progress, provide location updates.

8.1 Customer Module

The customer module supports account access, order creation, order visibility, and delivery tracking. Customer-facing order views are scoped to the authenticated customer's own orders.

8.2 Administrator Module

The administrator module provides operational oversight, including order management, delivery-agent assignment, customer management, and reporting screens.

8.3 Delivery Agent Module

The delivery-agent module focuses on assigned work. Agents can access assigned orders and progress an order through supported delivery statuses.

8.4 Notifications and Reports

The project includes notification-related backend routes/utilities and a notification interface component. Reporting screens provide an administrative view of delivery/order information.

9. Order Lifecycle and Business Workflow

The principal workflow begins when a customer creates an order. The administrator reviews the order and assigns a delivery agent. The agent collects the order, progresses it through transit, and marks it delivered. During an active delivery, location and status updates are made available to the tracking experience.

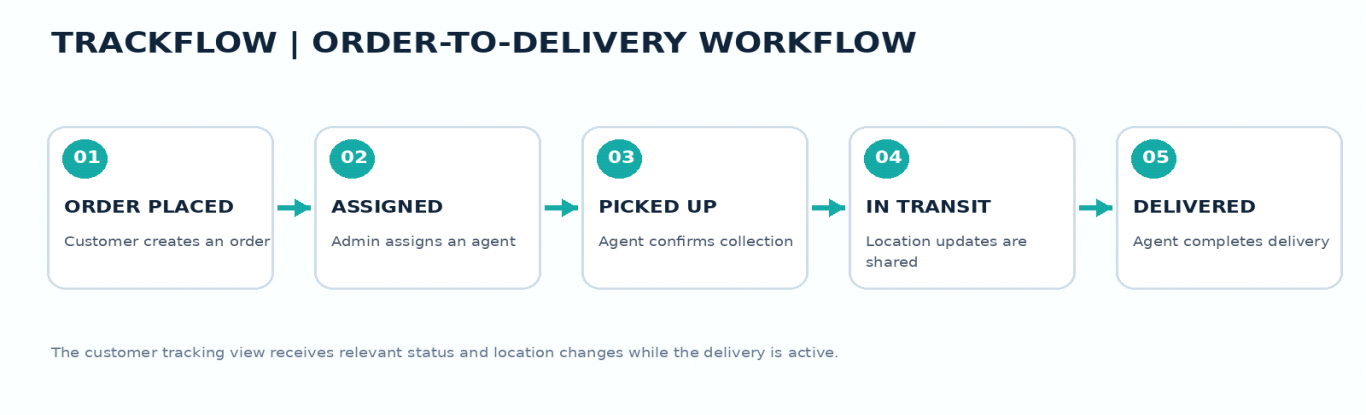


Figure 2. High-level order-to-delivery workflow.

Stage	Actor	System behavior
Order placed	Customer	A new order is submitted and stored.
Agent assigned	Administrator	The order is associated with a delivery agent.
Picked up	Delivery agent	The agent records collection.
In transit	Delivery agent / system	Progress and location updates are made available.
Delivered	Delivery agent	The order is marked complete and final status is recorded.

10. Real-Time Tracking with Socket.IO

Socket.IO supports real-time communication between the backend and connected frontend clients. A real-time connection allows the server to send an update when a relevant delivery event occurs, rather than requiring the user to refresh the page.

10.1 Conceptual Event Sequence

- The delivery agent is authenticated and connected to the application.
- During an active delivery, the agent's client provides a location or status update.
- The backend validates the update and associates it with the relevant order and agent.
- The backend persists applicable information, including location/status history where appropriate.
- The backend emits the relevant update to authorized connected clients.
- The customer's tracking interface receives the update and refreshes visible delivery information.

10.2 Benefits

Real-time updates reduce the delay between a delivery event and the customer's view of that event. They support a more responsive experience for administrators and agents. Access to real-time data must remain scoped so a user receives only information they are authorized to view.

10.3 Reliability Considerations

- Handle temporary network disconnections and reconnects.
- Re-fetch the latest order state after reconnecting.
- Validate identity and order access before order-specific updates.
- Use server-side timestamps for authoritative event ordering.
- Balance update frequency and location precision against data volume.

11. Map and Location Visualization

The tracking interface presents delivery location information in a map-oriented view. The application uses location coordinates to represent delivery progress and provide geographic context for the order. The project author confirmed that map display is complete in the local application.

11.1 Location Data

A location record typically contains latitude, longitude, an associated order or agent reference, and a timestamp. The backend should validate that latitude and longitude are within their legal ranges and associate each update with the correct active delivery.

11.2 Map Display

The map interface provides a visual representation of the available delivery position and relevant pickup/drop context. The exact map provider is not named because it has not been confirmed in the project details available for this document. Add the exact provider and API configuration details after checking the frontend implementation.

12. Backend API and Application Structure

The backend is organized into configuration, middleware, routes, and utility modules. This separation keeps database connectivity, authentication, authorization, HTTP endpoints, and reusable notification logic distinct.

Path / module	Responsibility
backend/src/server.js	Initializes Express application and server-side integrations.
backend/src/config/db.js	Configures PostgreSQL connectivity.
backend/src/middleware/auth.js	Authenticates requests.
backend/src/middleware/authorize.js	Applies role-based access checks.
backend/src/routes/users.js	User-related API operations.
backend/src/routes/orders.js	Order retrieval and order-related operations.
backend/src/routes/adminCustomers.js	Administrative customer operations.
backend/src/routes/notifications.js	Notification-related API operations.
backend/src/routes/reports.js	Reporting endpoints.
backend/src/utils/notifications.js	Reusable notification logic.

12.1 Health Check

A backend health endpoint is available at /api/health. During development, it was used to confirm that the API and PostgreSQL connection were available.

12.2 Frontend Structure

The frontend uses React components and page modules. The application entry point and routing structure connect public pages, authentication pages, dashboard layouts, and role-specific screens. Reusable components support order forms, order details, editing, notifications, and homepage sections.

13. Security and Access Control

Security is important because delivery information includes customer details, order records, and location data. TrackFlow uses token-based authentication and role-aware authorization middleware to protect backend operations. Passwords are handled using a password-hashing library rather than being stored as plain text.

13.1 Implemented Measures

- JWT-based authentication.
- bcrypt-based password hashing.
- Authorization middleware for role-specific operations.
- Order visibility scoped by role: customers access their own orders, agents access assigned orders, administrators have broader operational visibility.
- Environment variables for sensitive runtime configuration; environment files should not be committed to a public repository.

13.2 Production Hardening

- Use HTTPS for deployed traffic.
- Apply strict CORS rules for the production frontend origin.
- Validate and sanitize user input on the backend.
- Use secure token storage and sensible expiry/refresh policies.
- Apply rate limiting and security headers.
- Protect location data and retain only what is needed.
- Keep secrets in hosting environment configuration, not source control.

14. Testing and Verification

The project was exercised locally during development. The project author confirmed that the customer, administrator, and delivery-agent workflows, Socket.IO real-time updates, and map/location display are complete. The table summarizes reported local verification; it does not claim a formal automated test suite or third-party audit.

Test area	Verification focus	Reported result
Customer workflow	Account access, order creation, own-order visibility, tracking.	Verified locally by author
Administrator workflow	Order management, agent assignment, customer/report screens.	Verified locally by author
Delivery-agent workflow	Assigned order visibility and status progression.	Verified locally by author
Order lifecycle	Placed → assigned → picked up → in transit → delivered.	Exercised during development
Real-time updates	Socket.IO status/location updates reach tracking view.	Verified locally by author
Map/location display	Location represented in tracking interface.	Verified locally by author
Database connectivity	Backend API communicates with PostgreSQL.	Health endpoint used during development

14.1 Suggested Evidence

- Homepage and login page screenshot.
- Administrator order list and assignment screenshot.
- Delivery agent assigned-order view screenshot.
- Customer tracking page with map visible.
- Screenshot showing a status update reflected in customer view.
- Database table/test record screenshot with sensitive information hidden.

15. Challenges and Learning Outcomes

Developing a real-time delivery application requires coordination across frontend state, backend business logic, database records, authentication, and socket connections. The project provided practical experience integrating these layers into a working full-stack application.

15.1 Technical Areas Addressed

- Designing role-aware routes and user experiences for three user types.
- Connecting Express to PostgreSQL and maintaining related order records.
- Representing the order lifecycle consistently.
- Combining REST API requests with Socket.IO events.
- Associating location updates with the correct order and delivery agent.
- Organizing the frontend into reusable pages, layouts, contexts, and components.
- Debugging integration issues across browser, API, database, and real-time layers.

15.2 Skills Practiced

The project demonstrates practical use of React, Node.js, Express routing and middleware, relational database design, authentication and authorization, real-time event communication, and local application testing.

16. Limitations and Future Enhancements

The current project is a locally verified application. Public deployment is a planned next step and is not claimed as completed in this report. Further improvements can strengthen operational readiness.

- Deploy the frontend and backend and configure production environment variables.
- Use managed PostgreSQL with backups and access controls.
- Add automated unit, integration, and end-to-end tests.
- Add structured logging, metrics, error monitoring, and alerts.
- Improve reconnect handling and recovery for real-time socket sessions.
- Define configurable location update intervals and retention policies.
- Add delivery analytics such as average delivery duration and on-time completion rate.
- Document the exact map provider and API configuration.
- Add CI/CD workflows and dependency/security scanning.

17. Conclusion

TrackFlow brings together order management, delivery-agent coordination, customer tracking, and real-time delivery updates in a single full-stack web application. Its React frontend, Node.js/Express backend, PostgreSQL database, and Socket.IO communication layer provide the core technical foundation for the delivery workflow.

The application supports three primary roles and a delivery lifecycle from order placement through assignment, pickup, transit, and delivery. The project author has confirmed that the principal local workflows, real-time tracking, and map display are complete. The next major release activity is deployment, followed by continued testing and operational hardening.